SNACKIFY

ApplyWizz Office Pantry & Inventory Platform

Complete End-to-End Project Documentation
with Architecture, Pseudocode & Future Roadmap

Field	Value
Document type	Technical + Product E2E Guide
Version	1.1
Generated	2026-11-05
Stack	React + Vite + Express + Supabase + OpenAI
Audience	Engineers, Product, Ops, UX, Leadership
Adds in v1.1	UI/UX friction, Webhooks deep-dive, MNC comparison

Confidential — for internal ApplyWizz / Snackify team use.

Table of Contents

1. Executive Summary
2. Goals & Problem Statement
3. Technology Stack
4. High-Level Architecture
5. Repository Layout
6. User Roles & Access Matrix
7. Core Modules (with Pseudocode)
7.1 Authentication & MFA
7.2 Cafeteria Ordering
7.3 Order Fulfillment Queue
7.4 Meal Booking & Tokens
7.5 Inventory & Daily Update
7.6 Bill OCR & Product Conversion
7.7 Manual Purchases (Telegram)
7.8 Predictive Ordering / Forecasts
7.9 Meal Check-in (Vendor)
7.10 Admin, Finance & Reports
8. End-to-End System Flows
9. Database Overview
10. Integrations, Webhooks & Scheduled Jobs
11. Security Model
12. Deployment & Local Setup
13. Testing Strategy
14. Known Risks / Technical Debt
15. UI/UX — Reducing End-User Process Friction
16. Enterprise Landscape (Wipro / Infosys / MNC class)
17. Future Enhancements (Prioritized TODOs)
18. Glossary

1. Executive Summary

Snackify (ApplyWizz Office Pantry) is a full office facility operations platform. It covers pantry inventory, employee cafeteria ordering, meal booking with cabin token printing, AI-assisted bill extraction, Telegram manual purchases, predictive ordering, and role-based admin/finance workflows.

Employees order snacks and book meals; office boys fulfill queues and print tokens; facility managers keep stock accurate; finance verifies spend; leadership governs users, conversions, and forecasts. The system is domain-gated to ApplyWizz emails with mandatory TOTP MFA.

What the product delivers

- Real-time pantry/cafeteria stock with daily updates and transaction audit trail
- Employee cafeteria cart → confirming → pending → in_progress → done pipeline
- Shift-aware meal booking, token generation, and cabin print agent
- Invoice OCR + Product Conversion Master (safe AI fallback, idempotent stock apply)
- Telegram bot for register → extract → confirm → approve manual purchases
- Monday predictive order suggestions from 6-week consumption history
- Vendor meal check-in, guest meals, support tickets, push notifications

2. Goals & Problem Statement

Problems solved

- Paper / chat-based pantry requests with no stock truth
- Manual invoice entry errors and wrong unit conversions (boxes vs cups)
- No visibility for employees on order status
- Meal counts unknown until kitchen cutoff → waste or shortage
- Finance lacks linked spend vs inventory movements
- Office ops fragmented across Teams, Telegram, and spreadsheets

Product goals

- Single source of truth for stock, orders, meals, and bills
- Role-correct UX (staff never see raw finance/stock internals)
- AI to accelerate OCR and learning — never to auto-publish unsafe menu items
- Print and fulfill from Realtime events with minimal ops friction

3. Technology Stack

Layer	Technology	Notes
Frontend	React 18 + Vite + Tailwind	Role-gated routes in App.jsx
Backend	Node.js + Express	JWT auth middleware, service-role DB
Database	Supabase PostgreSQL	Migrations 0001–0058+, RLS + RPCs
Auth	Supabase Auth + TOTP MFA	Enrollment OTP via Microsoft Graph
AI	OpenAI GPT-4o / GPT-4o-mini	Bills, purchases, summaries, conversion
Print	print-agent + office-print-gateway	Realtime listeners + ESC/POS
Storage	Supabase Storage	Bill PDFs/images

Layer	Technology	Notes
Deploy	Vercel (FE) + Render/Railway (API)	Cron via backend + pg_cron
Tests	node:test + Playwright	Unit + E2E smoke/flows

4. High-Level Architecture

Browser clients hit the React app. Writes and privileged reads go through the Express API with a Bearer JWT. The API validates the user, loads profile.role, and talks to Postgres with the service role. The print agent listens to Supabase Realtime for order/meal print jobs. External systems (Teams, Telegram, Graph mail) are outbound or webhook-inbound.

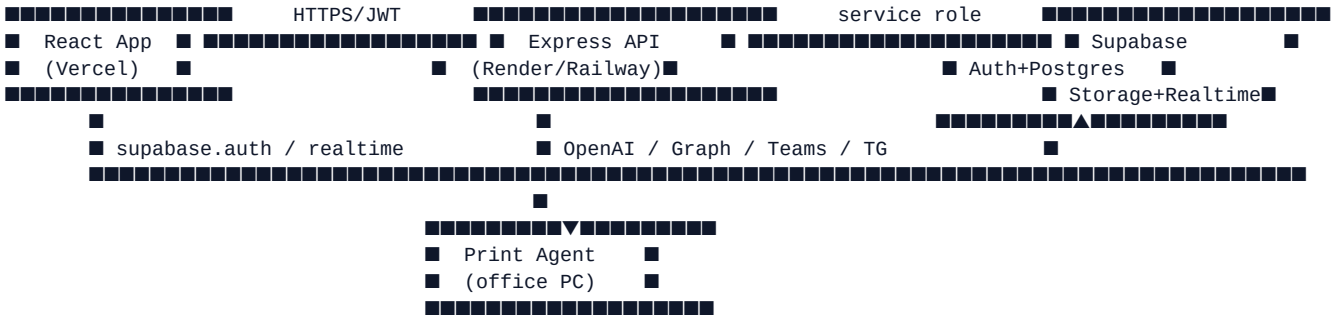

Figure 1 — Component topology

Request path (simplified)

```
FUNCTION handle_api_request(req):
    IF path is public webhook OR /api/auth OR /health:
        process without user JWT (use shared secret / flow rules)
    ELSE:
        user = supabase.auth.getUser(req.Authorization)
        profile = load profiles where id = user.id
        IF role not allowed for route:
            RETURN 403
        EXECUTE business logic with service-role client
        RETURN JSON
```

5. Repository Layout

Path	Purpose
frontend/	React UI — pages, auth hooks, API client
backend/	Express server, routes, libs, unit tests
supabase/migrations/	Schema evolution (products → vendor invites)
supabase/seed/	Starter product catalog
print-agent/	Desktop agent printing orders & meal tokens
office-print-gateway/	Receipt design + print gateway helpers
tests/	Playwright E2E
docs/	PRDs, architecture, this PDF
my-agent/	Eve agent scaffold (future ops assistant)
api/cron/	Hosted cron entrypoints (e.g. night shift report)

6. User Roles & Access Matrix

Role is stored on **profiles.role**. After login, RoleHome routes users to a default home. MFA AAL2 is required for most roles (leadership may use admin password path; impersonation is special-cased).

Role	Home	Primary job
leadership	/dashboard	Super-admin: users, bills, forecasts, everything
facility_manager	/dashboard	Stock health, daily update, ops
finance	/dashboard	Spend reports, bill payment approval
office_boy	/queue	Fulfill orders, print meal tokens, upload bills
staff	/request	Order cafeteria items, book meals
vendor	/vendor/meal-check-in	Meal check-in for catering vendor

Page access (summary)

Capability	Lead	FM	Fin	OB	Staff
Dashboard / Finance	Y	Y*	Y	N	N
Cafeteria order	Y	Y	Y	Y	Y
Order queue	Y	Y	N	Y	N
Meal booking / My Meal Box	Y	Y	Y	Y	Y
Meal token dashboard	Y	Y	N	Y	N
Bill upload	Y	Y	Y	Y	N
Bill approval	Y	N	Y	N	N
Daily stock update	Y	Y	N	N	N
Admin users	Y	N	N	N	N
Conversion master approve	Y	N	N	N	N

* Facility Manager dashboard/stock; Finance page is Accounts-focused.

7. Core Modules (with Pseudocode)

Each subsection describes purpose, key files, and language-agnostic pseudocode for the main algorithm. Pseudocode mirrors production intent, not every edge case.

7.1 Authentication & MFA

Key files: frontend Login.jsx, useAuth.js; backend routes/auth.js, otpService.js, microsoftGraph.js. First-time users enroll via email OTP then authenticator QR; returning users verify TOTP only.

```
PROCEDURE start_login(email):
    ASSERT email ends with allowed domain (@applywizz.ai or vendor domains)
    user = find auth user by email
    IF user has verified TOTP factor:
        RETURN { nextStep: "authenticator" }
    ELSE:
        RETURN { nextStep: "otp" }

PROCEDURE enroll_first_time(email):
    otp = generate_6_digit(); store hashed otp with expiry
    send_mail_via_graph(to=email, body=otp)
    ON verify_otp(email, code):
        create supabase user if missing
        factor = mfa.enroll(TOTP)
        RETURN qr_uri + enrollmentTransactionId
    ON verify_totp_enrollment(code):
        mfa.challengeAndVerify(factor, code)
        RETURN access_token + refresh_token

PROCEDURE login_existing(email, totp_code):
    session = generateLink + verifyOtp (server-side)
    mfa.challengeAndVerify(totp_code)
    RETURN tokens; frontend calls supabase.auth.setSession()
```

7.2 Cafeteria Ordering

Key files: Cafeteria.jsx, LiveTracking.jsx; cafeteria.js, requests.js. Catalog excludes items with visible_to_employees=false. Cart currently submits one POST per line item into status=confirming, then confirm moves to pending.

```
FUNCTION list_cafeteria_items():
    RETURN items WHERE available=true AND visible_to_employees <> false
    ENRICH with virtual drinks only if recipe + ingredients allow

PROCEDURE place_order(user, cart, location, note):
    created = []
    FOR EACH line IN cart:                    # today: sequential; future: atomic batch
        ASSERT stock_servings sufficient
        deduct stock_servings / stock_today
        INSERT request(status="confirming", item, qty, cabin, delivery_mode)
        created.append(request_id)
    SHOW success; NAVIGATE to tracking
    FOR EACH id IN created:
        confirm(id) → status = pending|placed
        fire push + Teams notification side effects

STATE MACHINE request:
    confirming → pending → in_progress → done
    confirming → cancelled (within cancel window)
    in_progress → ready_for_pickup → done
```

7.3 Order Fulfillment Queue

Office boy home is /queue (RequestQueue). Visible work is mainly non-confirming orders from v_request_queue. Print-agent may listen for confirming→pending and print receipts.

```
PROCEDURE office_boy_fulfill(request_id):
```

```
    ACCEPT → status = in_progress
    UPDATE prep step (preparing / on_the_way)
    IF delivery_mode = self_pickup:
        status = ready_for_pickup
        WAIT employee collect → done
    ELSE:
        mark delivered → done
```

7.4 Meal Booking & Tokens

Key files: MealBooking.jsx, MyMealBox.jsx, MealTokenDashboard.jsx; meals.js, mealPrint.js, cron.js, print-agent. Cutoffs depend on shift. Cron assigns token numbers and creates meal_print_jobs staggered by cabin.

```
PROCEDURE book_meal(user, date, choice):  # veg / non-veg / skip (+ options)
    window = compute_booking_window(user.shift, now_IST)
    IF now outside window: REJECT with cutoff message
    UPSERT meal_bookings(user, date, choice) UNIQUE per user/date

PROCEDURE cron_generate_tokens(working_day ~10:59 IST):
    bookings = today's non-skip bookings
    GROUP BY cabin
    FOR EACH cabin group:
        assign sequential token numbers
        INSERT meal_print_jobs(status=queued) with stagger delay
    print_agent Realtime → print cabin tickets → mark completed

FUNCTION my_token(user):
    RETURN token card OR "Token pending"
```

7.5 Inventory & Daily Update

Heart of pantry accounting: POST /api/inventory/daily-update. Diffs new counts vs current stock and writes add/remove transactions with unit_cost captured.

```
PROCEDURE daily_update(facility_manager, counts[]):
    FOR EACH {product_id, new_stock, optional price} IN counts:
        current = inventory.current_stock
        IF new_stock == current: CONTINUE
        delta = new_stock - current
        IF delta > 0:
            INSERT transaction(type="add", qty=delta, unit_cost, total_cost)
        ELSE:
            INSERT transaction(type="remove", qty=abs(delta), ...)
        UPDATE inventory SET current_stock=new_stock, last_updated_by=fm
    REFRESH dashboard views (stock_status, days_of_cover)
```

7.6 Bill OCR & Product Conversion Master

Office boy uploads invoice → Vision extracts lines → processInvoiceItems matches aliases in product_conversion_master. Unmatched lines get AI suggestions forced to pending/non-orderable. Leadership must resolve pending_review before Admin Verified applies stock via apply_bill_item_stock() (idempotent UNIQUE ledger).

```
PROCEDURE extract_and_convert(file, uploader):
    ASSERT not duplicate invoice fingerprint
    lines = openai_vision_extract(file)
    SAVE bill_uploads + bill_items
    ASYNC processInvoiceItems(lines):
        FOR EACH line:
            name = normalize(line.name)
            master = alias_match(product_conversion_master, name)  # GIN
            IF master:
                converted = qty * units_per_purchase_unit
                status = master_match OR no_stock (finance/equipment/supply)
            ELSE:
                batch unmatched for AI
        AI_batch(unmatched) → suggestions WITH forced:
            employee_orderable=false, visible_to_employees=false, pending_review
        IF AI fails: keep pending_review (upload still succeeds)
```

```
        saveConversions(...)

PROCEDURE approve_bill_admin_verified(bill_id):
    IF any item conversion_status = pending_review: RETURN 409
    FOR EACH eligible item:
        CALL apply_bill_item_stock(item_id)  # atomic RPC
            IF already in bill_stock_applications: RAISE ALREADY_APPLIED
            INSERT ledger row
            cafeteria_items.stock_servings += converted_quantity
            mark conversion_status = applied
    inventory_sync_status = complete
```

7.7 Manual Purchases (Telegram)

```
ON telegram_update:
    IF /register: map chat_id → profiles user; REPLY ok
    ELSE IF unregistered: ask to register; RETURN
    BUFFER text + photos for ~2 minutes
    EXTRACT with purchase AI
    INSERT manual_purchases
    RUN 3-step confirmation (name, qty, price)
    IF clear AND auto-approvable:
        applyPurchaseToInventory using conversion master lookup
        # never auto-create cafeteria SKUs
    ELSE:
        status = pending_review → finance/FM review UI
```

7.8 Predictive Ordering / Forecasts

Advisory only — never auto-orders. Monday cron (and leadership on-demand) reads ~6 weeks of remove transactions.

```
PROCEDURE run_weekly_forecast():
    FOR EACH active product:
        weeks = bucket_usage(last 6 weeks of remove txs)
        IF weeks_with_data >= 2:
            predicted = 0.6*avg(recent2) + 0.4*avg(older)
            basis = "history"
        ELSE:
            predicted = daily_usage * 5
            basis = "estimate"
        suggested = max(0, min(predicted - current_stock, max_safe_order))
        UPSERT product_forecasts
    OPTIONAL telegram digest to leadership for order_soon items
```

7.9 Meal Check-in (Vendor)

Vendor role lands on /vendor/meal-check-in. Backend feature module backend/src/features/mealCheckin provides QR/panel flows so catering vendors can confirm meal service attendance against bookings/checkins tables.

```
PROCEDURE vendor_checkin(token_or_qr, meal_slot):
    booking = resolve_booking(token_or_qr)
    ASSERT vendor authorized for that service
    INSERT meal_checkins(booking_id, checked_at, operator)
    UPDATE attendance aggregates for reports
```

7.10 Admin, Finance & Reports

- Admin: invite users, change roles, preferred names, forecast panel, MFA reset tools
- Finance: monthly spending views from add transactions + bill approval status
- Reports/Audit: operational and AI summary endpoints under /api/reports
- Support tickets, guest meals, tokens/cafeteria card pricing as supporting modules

8. End-to-End System Flows

8.1 Employee snack order (happy path)

```
Staff login (TOTP) → Onboarding if needed → /request
Select items → cart → confirm location
API creates confirming requests + stock deduct
Confirm → pending → Office Boy queue sees order
Print agent may print receipt
OB accepts → prepares → delivers → done
Staff tracks on /track/:id and sees /orders history
```

8.2 Meal day for an employee

```
Book on /meals before shift cutoff
Cron generates tokens + print jobs mid-morning
Employee opens /my-meal-box for token
OB uses /meal-token-dashboard for cabin batches
Vendor may check in diners via meal-check-in
Reviews / attendance reports available to ops
```

8.3 Invoice → stock (happy path)

```
OB uploads bill on /bills (or Teams/Telegram/webhook)
AI extract → bill_items saved → conversion async
Leadership reviews /bills/approve
Resolve any pending_review via master link / classify
Admin Verified → apply_bill_item_stock per line
Employees see updated stock_servings on cafeteria
Finance may separately set payment approval_status
```

8.4 Morning facility loop

```
FM opens Dashboard (alerts, days-of-cover)
Daily Update enters physical counts
Transactions recorded → Finance charts update
Low-stock alerts / digests to leadership
Monday forecast suggests replenishment quantities
```

9. Database Overview

Schema is migration-driven (0001_init through 0058+). Core domains:

Domain	Representative tables
Identity	profiles, enrollment_otps, login_transactions, mfa_reset_logs
Pantry	products, inventory, transactions, office_supplies
Cafeteria	cafeteria_items, requests, cafeteria_recipe_ingredients
Conversion	product_conversion_master, bill_*, bill_stock_applications
Meals	meal_bookings, meal_print_jobs, meal_checkins, meal_reviews
Purchases	manual_purchases, telegram_user_map, stock_takes
Planning	product_forecasts, v_latest_forecasts
Prefs	employee_cafeteria_preferences

Important invariant: bill stock apply is idempotent via UNIQUE(bill_item_id) on bill_stock_applications. Cafeteria visibility is server-enforced.

10. Integrations, Webhooks & Scheduled Jobs

Snackify is designed so humans do not re-type the same data across Teams, Telegram, email, and the web app. **Webhooks** are the machine-to-machine bridges: an external system POSTs an event into Snackify; Snackify validates a shared secret and runs the same business logic as a manual UI action.

Integration	Direction	Use
Microsoft Graph Mail	Outbound	Enrollment OTP emails
Microsoft Teams	Outbound	Order / ops Adaptive Cards (Power Automate URL)
Power Automate bill webhook	Inbound	Bill attachments → /api/bills/webhook
Telegram Bot webhook	In/Out	Manual purchases, digests, alerts
OpenAI	Outbound	OCR, NLP, conversion suggestions, summaries
Web Push	Outbound	Order/meal reminders
Print Agent Realtime	Inbound listen	Orders & meal_print_jobs
Guest meal accept links	Inbound	Public /api/guest-meal secured by URL secret

10.1 Why webhooks matter (ops clarity)

- Office Boy can drop an invoice photo in Teams → Power Automate → Snackify, without opening Bill Upload
- Telegram purchase messages become structured stock rows without a second data-entry form
- Cron/machine callers hit /api/cron/* with CRON_SECRET — same pattern as webhooks (secret, not user JWT)
- Without webhooks, every channel becomes a separate spreadsheet and stock drifts

10.2 Bill webhook (Power Automate / Make / Zapier)

```
ENDPOINT:  POST /api/bills/webhook?key=<BILL_WEBHOOK_KEY>
AUTH:      Shared secret query key (NOT employee JWT)
BODY:      multipart file OR JSON with file URL + metadata
FLOW:
  1. Validate webhook key → 401 if wrong
  2. Store file in Supabase Storage bills/ (power-automate/...)
  3. OpenAI Vision / PDF extract → vendor, invoice #, line items, totals
  4. Duplicate invoice check (block double stock)
  5. INSERT bill_uploads + bill_items
  6. ASYNC processInvoiceItems + saveConversions  (same as manual upload)
  7. RETURN ok; leadership still reviews on /bills/approve before stock apply

NOTE: Microsoft retired classic Office 365 Connector webhooks (webhook.office.com).
      Teams outbound must use a Power Automate HTTP trigger URL (see teams.js).
```

10.3 Telegram webhook

```
ENDPOINT:  POST /api/telegram/webhook?key=<TELEGRAM_WEBHOOK_KEY>
AUTH:      Shared secret; Telegram delivers Update JSON
FLOW:
  /register → map chat_id to profiles user
  text + photos buffered (~2 min) → AI extract purchase
  3-step confirm in chat → auto-approve OR pending_review
  applyPurchaseToInventory via conversion master (never invent cafeteria SKUs)
IDEMPOTENCY: ignore Telegram retries after restart where already processed
```

10.4 When you need a webhook vs the UI

Situation	Prefer
One-off bill while at desk	UI /bills upload

Situation	Prefer
Bills arrive in Teams channel all day	Power Automate → bill webhook
Purchase made at kirana with phone only	Telegram webhook bot
Scheduled meal print / forecast / digest	Cron + CRON_SECRET
Employee orders tea	UI cafeteria (no webhook)

Typical cron / scheduled work

- Meal booking reminders / skip reminders / confirmation mails
- Meal print job generation (~10:59 IST working days)
- Night-shift meal booking reports
- Stock alerts & daily digests
- Weekly forecast (Mondays)
- Cabin sync cron

11. Security Model

- Domain-gated signup; default role staff via handle_new_user trigger
- Mandatory TOTP MFA (AAL2) for standard users
- Express authMiddleware validates JWT; requireRole on mutating routes
- Service role used only server-side after authz checks
- Webhook routes protected by shared secrets (not user JWT)
- AI conversion cannot publish employee-visible/orderable items without approval
- Inactivity lock on frontend; leadership impersonation is audited carefully
- Global API rate limit (~120 req/min) — tune per-route for polling pages

12. Deployment & Local Setup

Local (developer)

```
1. Create Supabase project; run migrations 0001 → latest; seed products
2. backend: copy .env.example → .env (SUPABASE_URL, SERVICE_ROLE, OPENAI, ...)
   npm install && npm run dev          # :4000  /health → ok
3. frontend: .env.local with VITE_SUPABASE_URL, ANON_KEY, VITE_API_BASE_URL
   npm install && npm run dev          # :5173
4. Promote yourself: UPDATE profiles SET role='leadership' WHERE email=...
5. Optional: print-agent on office machine with Supabase Realtime credentials
```

Production shape

- Frontend: Vercel (snackify.applywizz.ai / inventory-ashen-theta.vercel.app)
- Backend: Render/Railway (e.g. inventory-vgor.onrender.com)
- Database/Auth/Storage: Supabase
- Set ALLOWED_ORIGINS, CRON_SECRET, webhook keys, Graph & Telegram secrets

13. Testing Strategy

Layer	Location	Focus
Unit	backend/tests/*.js	Auth, conversion math, admin, attendance
Product conversion	productConversion.test.js	18 cases: aliases, AI safety, idempotency
E2E	tests/e2e/*.spec.js	Smoke, meal-box, purchases, menu cleanup
Manual	QA checklists in docs	MFA refresh, multi-item cart, 429 soak

14. Known Risks / Technical Debt

- Multi-item cart is non-atomic / non-idempotent (partial success risk)
- Print-agent DB fallback can bypass backend confirmation side effects
- Tracking page polling + global rate limit → 429 under sustained use
- Onboarding 'No Snacks' should be mutually exclusive with snack picks
- Session persistence / AAL2 gating regressions appear as login loops
- Telegram depends on live webhook, mapping, and storage configuration

15. UI/UX — Reducing End-User Process Friction

Many employees feel Snackify has a **high process load**: MFA, onboarding, cabin pick, cart confirm, tracking, meal cutoffs, tokens. The backend is correctly strict for stock and security — but the **felt effort** for a cup of tea must stay near consumer apps (Swiggy / Zomato mental model), not ERP forms. This section separates what must stay hard (security/stock) from what should feel invisible.

15.1 Current end-user journey (felt steps)

```
TODAY — Staff ordering a snack (high perceived process):
  1. Open app / remember URL
  2. Email → wait / Authenticator TOTP
  3. First-time onboarding (prefs, cabin, snacks)
  4. Find cafeteria page (or land correctly by role)
  5. Browse categories → add items
  6. Open cart → pick location → delivery mode → note
  7. Submit (multi API calls) → success toast
  8. Tracking page polls status → may see busy/429
  9. Optional: cancel window, history later

TODAY — Meal:
  Book before cutoff → wait for token → open My Meal Box → show token / cabin print

GOAL — Same security, fewer conscious decisions:
  Open → (biometric/session already warm) → 1-tap re-order → track in one calm screen
```

15.2 Pain → UI/UX change map

End-user feeling	Root cause	UI/UX change
"Too many steps for chai"	Cart + location + confirm every time	1-tap Reorder last; remember cabin; skip note unless needed
"I got logged out again"	AAL2 / session restore gaps	Warm session restore spinner; clear MFA resume; no silent bounce
"Did my order go through?"	Multi-item partial POST + toast	Single checkout progress; atomic success/fail; order ID first
"Tracking feels stuck / busy"	5s polling + rate limit	Realtime status; soft offline banner; less chrome noise
"Meal token pending forever"	Cron/print lag unexplained	Timeline: Booked → Token soon → Printed; ETA copy
"Onboarding is a form wall"	All prefs upfront	3 short screens; defaults; edit later in Settings
"Office Boy UI is dense"	Queue + tokens + bills mixed mental load	Role home = one primary action; secondary in More
"Finance vs stock approval confusing"	Two approval concepts on bills	Plain labels: Pay bill vs Add to pantry stock

15.3 Design principles for Snackify UX

- One job per screen — staff: Order; OB: Next order; FM: Stock health
- Defaults over decisions — cabin, delivery mode, last cart remembered
- Progressive disclosure — advanced prefs/recipes/conversion stay out of staff path
- Honest status language — Booked / Preparing / On the way / Done (no jargon)
- Motion for confirmation, not decoration — press feedback, success settle
- Empty & error states teach next action — not only red text
- Role-home clarity — never dump Dashboard chrome on staff
- Mobile thumb-zone — OB and staff primarily on phones between cabins

15.4 Target simplified flows (pseudocode)

```
PROCEDURE staff_fast_order():
    IF warm_session: skip login chrome
    IF last_order exists:
```

```
        SHOW "Order again: Assam Tea ×1 → Balaji Cabin"  [primary CTA]
        ON tap: place_atomic_cart(clone last) → open calm tracking
    ELSE:
        SHOW personalized top items (3–6) + Search
        ADD → sticky cart bar with one Confirm button
        Confirm uses saved cabin; ask location ONLY if missing

PROCEDURE meal_calm_status():
    SHOW single card: meal choice + cutoff countdown OR token big number
    IF token pending: "Kitchen printing cabin tickets ~11:00" + progress dots
    HIDE secondary ops links from staff

PROCEDURE reduce_onboarding_friction():
    Screen1: Preferred name + cabin (required)
    Screen2: Reminders on/off (default on)
    Screen3: Optional snack likes (skip allowed)
    SAVE; never block cafeteria if prefs incomplete beyond cabin
```

15.5 UI surfaces to redesign (priority)

- Cafeteria.jsx checkout sheet — fewer fields, sticky confirm, reorder
- LiveTracking.jsx — realtime, quieter chrome, clear cancel window
- Onboarding.jsx — shorter; exclusive No Snacks
- Login.jsx — step indicator (Email → OTP/TOTP → In); residual session messaging
- RequestQueue.jsx — card = next action only; filters secondary
- BillApproval.jsx — conversion badges in plain English + Pay vs Stock verbs
- MyMealBox / MealTokenDashboard — staff vs OB information density split

16. Enterprise Landscape — Do MNCs Have This?

Yes. Large IT / BPO / manufacturing campuses (Wipro, Infosys, TCS, Accenture, Cognizant, Google/Microsoft campuses, and many GCCs in India) run **the same problem domain**: cafeteria, pantry, visitor meals, facility spend, and attendant workflows. They rarely ship one small React app — they buy or build **suite platforms**. Snackify is the mid-market / single-campus equivalent of those high-end facility & workplace stacks.

16.1 What big companies typically use

Capability	MNC / enterprise pattern	Snackify today
Employee cafeteria / snacks	Sodexo / Compass / local F&B apps; often badge/QR + wallet	Catalog + cart + tokens/coins + tracking
Meal booking / attendance	HRMS + cafeteria vendor portals; cutoff windows; cabin/workstation	Shift cutoffs, tokens, print-agent, vendor check-in
Pantry / consumables inventory	SAP MM / Oracle Inventory / ServiceNow FM modules	products + inventory + daily update + transactions
Invoice → stock	AP OCR (Coupa, Tipalti, SAP Concur) + GRN in ERP	Bill OCR + Product Conversion Master + idempotent apply
Field purchases	P-card + WhatsApp/Teams bots into procurement	Telegram bot + manual purchase review
Demand planning	ERP MRP / Blue Yonder style forecasts	6-week weighted forecast (advisory)
Identity	Okta / Azure AD / Ping + conditional MFA	Supabase + Graph OTP + TOTP MFA; domain gate
Notifications	Workplace / Teams / email / SMS gateways	Teams cards, Telegram, web push, mail cron
Integrations	Enterprise service bus / iPaaS (MuleSoft, Boomi)	Webhooks (bills, Telegram) + cron secrets

16.2 High-end functionalities MNCs expect (gap map)

These are the **enterprise-grade** features leadership at Wipro/Infosys-scale usually asks for. Snackify already covers the campus core; items below are the ladder to that class of system.

High-end capability	Why MNCs need it	Snackify status
SSO (Azure AD / Okta) + SCIM provisioning	10k+ users; joiner/leaver automation	Partial (email domain); SCIM not built
Badge / RFID / QR meal redemption	Cafeteria throughput & fraud control	Tokens + vendor check-in; badge TBD
Multi-campus / multi-tower tenancy	Wipro has many SEZs	Single-tenant campus model
Budget centers & cost allocation	Chargeback to projects/cost centers	Finance spend views; cost center TBD
Vendor SLA & GRN matching	3-way match PO–GRN–Invoice	Invoice OCR + stock apply; PO match TBD
Contracted menu & allergen labels	Compliance / wellness	Menus evolving; allergens TBD
IoT fridge / vending telemetry	Unmanned pantry	Not in scope yet
Workplace experience suite embed	ServiceNow / Microsoft Viva tiles	Standalone web app
Audit, SOX-friendly immutable ledgers	Internal audit / ISO	tx + bill_stock_applications strong start
24×7 observability & SLOs	NOC style ops	Minimal; needs tracing/alerts
iPaaS webhooks + event bus	Connect HRMS, ERP, LMS	Bill + Telegram webhooks present
Mobile attendant apps (offline)	Housekeeping / pantry boys	Responsive web; offline TBD

16.3 Positioning statement

Snackify is not trying to replace SAP tomorrow. It is a **focused workplace F&B + pantry operating system** for ApplyWizz-scale offices that already matches the day-to-day loops Infosys/Wipro campuses solve with heavier

suites — ordering, meals, stock, bills, bots — with AI conversion safety that many legacy ERPs still lack. The roadmap (Section 17) climbs toward MNC expectations: SSO, multi-site, PO/GRN, cost centers, and calmer UX so end users feel consumer-grade speed.

```
ENTERPRISE MATURITY LADDER (Snackify):
  L1 Campus ops     ■ orders, meals, inventory, bills, Telegram  (current)
  L2 Trust & UX     → atomic checkout, realtime track, SSO feel, low friction UI
  L3 Finance grade  → PO/GRN match, cost centers, stronger audit exports
  L4 Multi-site     → towers/campuses, vendor networks, badge redeem
  L5 Platform       → event bus, embed in Workplace/ServiceNow, IoT pantry
```

17. Future Enhancements — Prioritized TODO List

Ordered by impact × risk reduction × end-user calm. Treat P0 as current sprint candidates, P1 as next planning cycle, P2 as growth / MNC ladder, P3 as backlog.

P0 — Critical stability (do first)

1. Atomic multi-item cart API with idempotency key
Why: Stops partial orders, duplicate submits, false success toasts.

2. Single owned confirmation path for orders
Why: Remove print-agent direct DB status writes; keep Teams/push side effects consistent.

3. Fix session persistence / refresh AAL2 regressions
Why: Login loops after OTP or page refresh block meals and ordering.

4. Route-level rate limits + cheaper LiveTracking polling/Realtime
Why: Eliminates 429s after ~2 minutes on tracking tabs.

5. BillApproval UI for conversion_status / AI suggestions
Why: Backend blocks pending_review; ops need first-class resolve/link/classify UX.

6. Calm checkout UX: one Confirm, progress, clear order ID
Why: Cuts 'did it go through?' anxiety; pairs with atomic cart API.

P1 — High value reliability, UX calm & ops

1. 1-tap Reorder + remembered cabin/delivery defaults
Why: Biggest reduction in staff process for daily chai/snacks.

2. Shorter onboarding (3 screens) + No Snacks exclusivity
Why: Less form-wall; cleaner preference data for reminders/AI.

3. Realtime tracking + quieter LiveTracking chrome
Why: Removes busy/429 feeling; consumer-grade status card.

4. Realtime order queue for office boy (no manual refresh)
Why: Faster fulfillment and fewer missed confirming→pending transitions.

5. Plain-English bill verbs: Pay bill vs Add to pantry stock
Why: Stops finance/stock approval confusion for leadership.

6. Coffee recipe management UI (grams-per-cup)
Why: Re-enable virtual espresso drinks safely via cafeteria_recipe_ingredients.

7. Telegram + bill webhook observability + retry queue
Why: Detect webhook failures; replay buffered purchases/invoices.

8. Structured logging/tracing for confirm, print, bill-apply, webhooks
Why: Cuts MTTR on production incidents (MNC-style ops readiness).

9. Facility Manager Forecast panel on Dashboard
Why: API already allows FM read; move suggestions closer to daily ops.

10. E2E coverage: multi-item cart, auth refresh, 429 soak, reorder
Why: Locks P0/P1 UX fixes against regression.

P2 — Product growth & MNC ladder (L3)

1. Azure AD / Microsoft Entra SSO (keep TOTP where required)
Why: Matches Wipro/Infosys identity expectations; fewer enrollment steps.

2. Hyperpure / vendor API automated PO suggestions + light GRN
Why: Turn forecasts into draft POs; step toward 3-way match.

3. Cost center / project chargeback on orders & bills
Why: Finance-grade allocation MNCs expect.

4. Slack/email channels for low-stock & expiry (beyond Telegram)
Why: Broader stakeholder coverage.

5. Consumption-trend analytics dashboards
Why: Category burn rates, cabin demand heatmaps, waste tracking.

6. Offline-capable daily update (PWA draft sync)
Why: Pantry counts when Wi-Fi is weak — attendant-app class.

7. Guest meal & visitor UX polish + reporting pack
Why: Better hospitality metrics for leadership.

8. Token/cafeteria card wallet enhancements + big meal-token timeline UI
Why: Clearer balances; removes 'token pending forever' feeling.

9. Meal attendance PDF/email packs automation
Why: Reduce manual report chasing for vendors and HR.

10. Webhook catalog page in Connections (keys rotated, last delivery, replay)
Why: iPaaS-lite ops UI so webhooks are operable, not tribal knowledge.

P3 — Backlog / MNC ladder L4–L5 experiments

1. Multi-campus / multi-tower tenancy
Why: Required when expanding beyond one ApplyWizz office.

2. Badge / RFID / QR cafeteria redemption at counter
Why: High-throughput dining hall pattern used on large campuses.

3. ServiceNow / Viva / Teams personal-app embed
Why: Meet employees where they already work.

4. Eve agent (my-agent/) production purpose & playbooks
Why: Natural-language ops assistant for leadership digests and triage.

5. Staging environment parity checklist
Why: Auth URLs, webhooks, Graph, Telegram, print-agent dry-run.

6. Advanced personalization (self-learning brain v2)
Why: Safer preference learning with explicit consent controls.

7. WhatsApp bill intake channel
Why: Parallel to Telegram for vendors who prefer WA.

8. IoT fridge / vending telemetry hooks
Why: Unmanned pantry experiments.

9. Cleanup legacy print-agent paths and dead mappers
Why: Reduce cognitive load for new contributors.

Suggested delivery sequence

```
Sprint A (P0): atomic cart + confirm ownership + session fix + calm checkout
Sprint B (P0/P1): rate-limit/realtime track + BillApproval verbs + reorder+cabin defaults
Sprint C (P1): short onboarding + OB queue realtime + webhook retries + E2E
Sprint D (P2): SSO + PO drafts + cost centers + webhook catalog UI
Sprint E (P3): multi-site / badge redeem / Workplace embed experiments
```

Priority checklist (printable)

Pri	TODO
P0	[ ] Atomic cart + idempotency
P0	[ ] Confirm path ownership (no agent DB bypass)
P0	[ ] Session / AAL2 persistence
P0	[ ] Rate limit + tracking polling / realtime
P0	[ ] BillApproval conversion review UI
P0	[ ] Calm checkout (one confirm + order ID)

Pri	TODO
P1	[ ] 1-tap Reorder + remembered cabin
P1	[ ] Short onboarding + No-Snacks exclusivity
P1	[ ] Quiet realtime LiveTracking
P1	[ ] Realtime OB queue
P1	[ ] Pay bill vs Add stock labels
P1	[ ] Coffee recipe admin UI
P1	[ ] Webhook retry + observability
P1	[ ] Structured tracing
P1	[ ] FM forecast panel
P1	[ ] E2E cart/auth/reorder tests
P2	[ ] Azure AD / Entra SSO
P2	[ ] Vendor PO draft + light GRN
P2	[ ] Cost center chargeback
P2	[ ] Webhook catalog in Connections
P2	[ ] Analytics + offline daily update
P2	[ ] Meal token timeline + wallet polish
P3	[ ] Multi-campus tenancy
P3	[ ] Badge / QR redemption
P3	[ ] Workplace / ServiceNow embed
P3	[ ] Eve agent / WhatsApp / IoT pantry

18. Glossary

Term	Meaning
AAL2	Authenticator Assurance Level 2 — TOTP MFA satisfied
stock_servings	Employee-facing converted units (cups, slices)
stock_today	Raw/purchase-oriented stock field
confirming	Brief pre-queue order state (~30s cancel window)
PCM	product_conversion_master — invoice→stock rules
OB	Office Boy — fulfiller role
FM	Facility Manager
IST	India Standard Time — meal cron schedule basis
RPC	Postgres function called via Supabase (e.g. apply_bill_item_stock)
Webhook	External system POSTs events with a shared secret (no user JWT)
iPaaS	Integration Platform as a Service (Power Automate, Boomi, MuleSoft)
GRN	Goods Receipt Note — warehouse confirmation against a PO
SSO	Single Sign-On (Azure AD / Okta) — one corporate login
Felt process	Number of conscious decisions a user notices (UX load)

Document control

This PDF (v1.1) was generated from the checked-in Snackify repository structure, README, architecture notes, role map, webhook routes, and server/router inventory. It covers technical flows, UI/UX friction reduction, webhook ops, and how Snackify maps to MNC facility platforms. Not a substitute for live runbooks or secrets. Regenerate via: `python docs/generate_e2e_pdf.py`

— End of document —